



ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL

SISTEMAS EMBEBIDOS

INTEGRANTES:

Baldeón Baque Iván

Martínez Méndez Jennifer

Terán Santana Francisco

Velasquez García Jordy

DOCENTE:

Solis Mesa Ronald David

JUNIO DE 2026

PERIODO ACADÉMICO

PAO 1 2026

Escuela Superior Politécnica del Litoral
Facultad de Ingeniería en Electricidad y Computación

Integrantes:

- Jennifer Amelia Martínez Méndez – jenammar@espol.edu.ec
- Iván Gonzalo Baldeón Baque -- ivagbald@espol.edu.ec
- Francisco Israel Terán Santana – fraister@espol.edu.ec
- Jordy Jerhel Velasquez García – jjvelasq@espol.edu.ec

Fecha de entrega: 10 de junio de 2026

Objetivo general:

Desarrollar el juego interactivo "Simón Dice" con tres niveles de dificultad, empleando un ATmega328P para el control visual de una matriz LED 8x8 y un PIC16F887 para la generación de eventos sonoros. Todo el sistema será programado en lenguaje C, establecerá comunicación entre ambos microcontroladores y será validado mediante simulación en Proteus.

Objetivos específicos:

- Controlar una matriz LED 8x8 mediante el microcontrolador ATmega328P para visualizar las secuencias del juego.
- Implementar los tres niveles de dificultad modificando parámetros de velocidad y complejidad en la ejecución del código.
- Configurar un canal de comunicación entre microcontroladores que garantice la sincronización de las salidas visuales y sonoras.

1. Descripción técnica del juego

La dinámica implementada es la clásica memoria secuencial, el juego sigue la siguiente lógica:

- a. El ATmega328P genera de forma aleatoria una secuencia de indicadores visuales en este caso flechas, mostrandolas con patrones específicos y sincronizados con los eventos del juego.
- b. Simultáneamente al patrón de la matriz 8x8 se emitirá un tono característico por cada inicio de ronda y cada vez que el jugador deba interactuar con los botones durante la secuencia de niveles.
- c. El jugador deberá memorizar la secuencia y replicarla en el orden exacto con ayuda de pulsadores.
- d. Si el usuario acierta, la secuencia aumenta un paso. Si no se logra acertar la secuencia, el ATmega328PP envía una señal de error al PIC16F887 para que este reproduzca la melodía de "Game Over" y el juego se reinicia.

El nivel del juego va progresivamente aumentando según la cantidad de patrones que el jugador pueda completar.

Para la arquitectura del sistema de procesamiento se emplean dos microcontroladores sincronizados mediante un canal de comunicación paralela.

- **Módulo de control y visualización (ATmega328P):** Este va a actuar como núcleo lógico del sistema. Se encarga de ser el cerebro del juego, procesa toda la lógica anteriormente descrita, genera las secuencias visuales en la matriz LED 8x8 y registra las entradas del jugador por medio de los pulsadores físicos. El código se desarrolla en C.
- **Módulo de audio (PIC16F887):** Funciona de manera esclava, se va a dedicar de forma exclusiva a la interpretación de comandos recibidos desde el ATmega328P para reproducir melodías con ayuda de un buzzer. De esta forma se vinculan de forma eficiente y sincronizada los efectos sonoros con los eventos del juego.

Para la dificultad del juego se tienen cinco niveles:

- **Nivel 1:** La velocidad de reproducción de la secuencia en la matriz LED es igual para cada nivel. El usuario cuenta con un tiempo amplio para presionar cada pulsador, ideal para comprender la dinámica.
- **Nivel 2:** Implementa la lógica estándar del juego. La velocidad de la animación y la dificultad van aumentando, ahora se presenta un patrón más la secuencia.
- **Nivel 3:** Diseñado con un menor margen de error. La velocidad con la que el ATmega328P muestra la secuencia es acumulativa, y la posibilidad para que el usuario ingrese la respuesta correcta o falle incrementa.
- **Nivel 4-5:** Para este punto el usuario ya debió haber comprendido la dinámica del juego y haber memorizado la secuencia progresiva, en caso de fallo en cualquier nivel sonará la melodía de game over o error, y el juego se reiniciará automáticamente al nivel 1 cambiando el patrón de la secuencia.

2. Simulación en Proteus

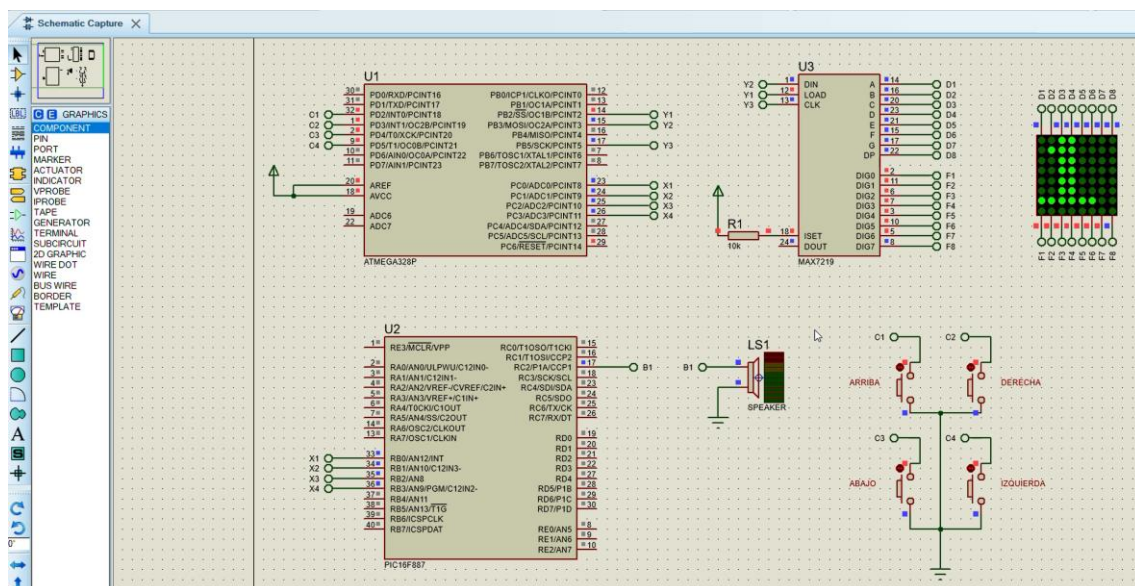


Figura 1. Indicador del nivel de juego.

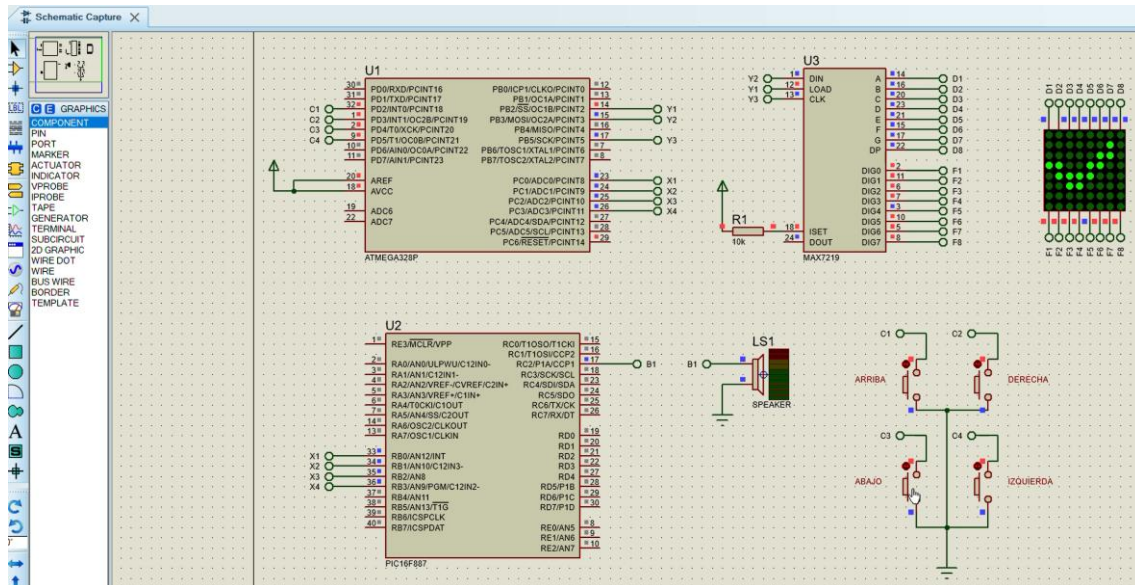


Figura 2. Si el jugador acierta se mostrará el símbolo de check al final.

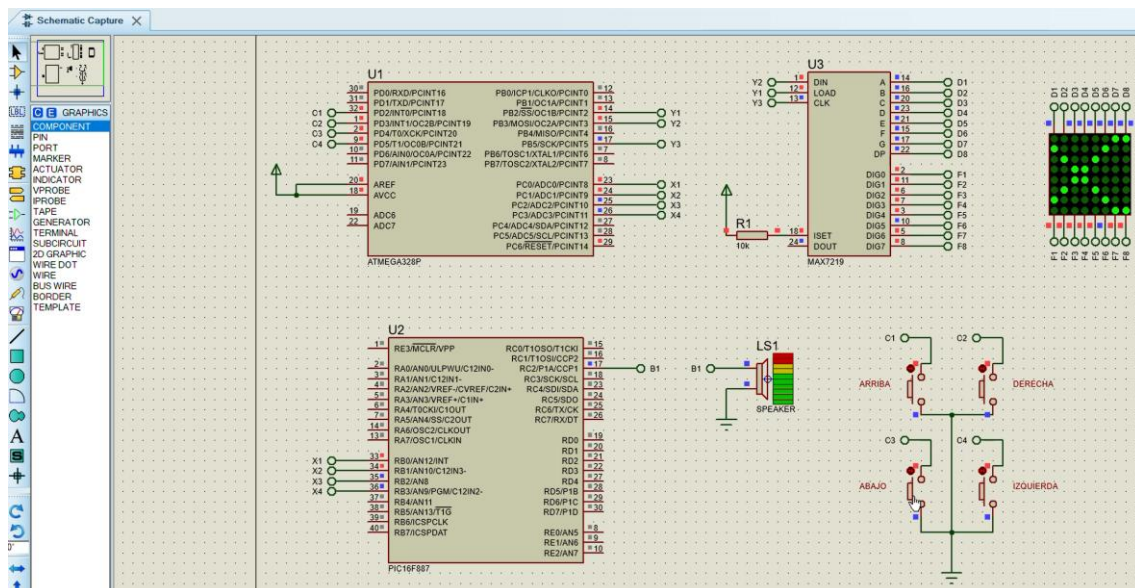


Figura 3. Si el jugador no logra acertar en la secuencia, le saldrá una equis y también su respectivo sonido.

3. Explicación del código

Código para ATmega328P (Matriz LED 8x8):

```
#include <avr/io.h>
#include <util/delay.h>
#include <stdlib.h>
```

```
#define DIN    PB3
#define LOAD   PB2
#define CLK    PB5
```

```
#define BTN1 PD2
#define BTN2 PD3
#define BTN3 PD4
#define BTN4 PD5
```

```
uint8_t nivel = 1;
uint8_t secuencia[8];

void enviarPIC(uint8_t dato)
{
    PORTC = (PORTC & 0xF0) | dato;

    _delay_ms(100);

    PORTC &= 0xF0;
}

void maxSend(uint8_t reg, uint8_t data)
{
    PORTB &= ~(1 << LOAD);

    for(int8_t i=7;i>=0;i--)
    {
        if(reg & (1<<i))
            PORTB |= (1<<DIN);
        else
            PORTB &= ~(1<<DIN);

        PORTB |= (1<<CLK);
        PORTB &= ~(1<<CLK);
    }

    for(int8_t i=7;i>=0;i--)
    {
        if(data & (1<<i))
            PORTB |= (1<<DIN);
        else
            PORTB &= ~(1<<DIN);

        PORTB |= (1<<CLK);
        PORTB &= ~(1<<CLK);
    }

    PORTB |= (1<<LOAD);
}

void maxInit()
{
    maxSend(0x09,0x00);
    maxSend(0x0A,0x08);
    maxSend(0x0B,0x07);
    maxSend(0x0C,0x01);
    maxSend(0x0F,0x00);
}

void clearDisplay()
{
    for(uint8_t i=1;i<=8;i++)
        maxSend(i,0);
}

void draw(uint8_t bmp[])
{
    for(uint8_t i=0;i<8;i++)
        maxSend(i+1,bmp[i]);
}

/* FLECHAS */

uint8_t up[8]={
    0x18,0x3C,0x7E,0x18,
    0x18,0x18,0x18,0x00};
```

```
uint8_t right[8]={
0x10,0x18,0x1C,0xFE,
0xFE,0x1C,0x18,0x10};

uint8_t down[8]={
0x18,0x18,0x18,0x18,
0x7E,0x3C,0x18,0x00};

uint8_t left[8]={
0x08,0x18,0x38,0xFE,
0xFE,0x38,0x18,0x08};

/* OK */

uint8_t okBmp[8]={
0x00,0x01,0x03,0x66,
0x3C,0x18,0x00,0x00};

/* ERROR */

uint8_t errBmp[8]={
0x81,0x42,0x24,0x18,
0x18,0x24,0x42,0x81};

/* NUMEROS */

uint8_t num1[8]={
0x18,0x38,0x18,0x18,
0x18,0x18,0x7E,0x00};

uint8_t num2[8]={
0x3C,0x66,0x06,0x0C,
0x18,0x30,0x7E,0x00};

uint8_t num3[8]={
0x3C,0x66,0x06,0x1C,
0x06,0x66,0x3C,0x00};

uint8_t num4[8]={
0x0C,0x1C,0x3C,0x6C,
0x7E,0x0C,0x0C,0x00};

uint8_t num5[8]={
0x7E,0x60,0x7C,0x06,
0x06,0x66,0x3C,0x00};

void mostrarNivel(uint8_t n)
{
    switch(n)
    {
        case 1: draw(num1); break;
        case 2: draw(num2); break;
        case 3: draw(num3); break;
        case 4: draw(num4); break;
        case 5: draw(num5); break;
    }

    _delay_ms(1000);

    clearDisplay();
}

void mostrarFlecha(uint8_t n)
{
    switch(n)
    {
        case 0: draw(up); break;
```



```

        case 1: draw(right); break;
        case 2: draw(down); break;
        case 3: draw(left); break;
    }
}

uint8_t leerBoton()
{
    while(1)
    {
        if(!(PIND & (1<<BTN1)))
        {
            _delay_ms(50);
            while(!(PIND & (1<<BTN1)));
            return 0;
        }

        if(!(PIND & (1<<BTN2)))
        {
            _delay_ms(50);
            while(!(PIND & (1<<BTN2)));
            return 1;
        }

        if(!(PIND & (1<<BTN3)))
        {
            _delay_ms(50);
            while(!(PIND & (1<<BTN3)));
            return 2;
        }

        if(!(PIND & (1<<BTN4)))
        {
            _delay_ms(50);
            while(!(PIND & (1<<BTN4)));
            return 3;
        }
    }
}

int main()
{
    DDRB |= (1<<DIN) | (1<<LOAD) | (1<<CLK);

    DDRC |= 0x0F;

    DDRD &= ~(1<<BTN1)|(1<<BTN2)|(1<<BTN3)|(1<<BTN4));

    PORTD |= (1<<BTN1)|(1<<BTN2)|(1<<BTN3)|(1<<BTN4);

    maxInit();

    srand(123);

    for(uint8_t i=0;i<8;i++)
        secuencia[i] = rand()%4;

    while(1)
    {
        uint8_t longitud = nivel + 3;

        mostrarNivel(nivel);

        switch(nivel)
        {
            case 1: enviarPIC(5); break;
            case 2: enviarPIC(6); break;
            case 3: enviarPIC(7); break;
        }
    }
}

```



```
        case 4: enviarPIC(8); break;
        case 5: enviarPIC(9); break;
    }

    enviarPIC(1);

    for(uint8_t i=0;i<longitud;i++)
    {
        mostrarFlecha(secuencia[i]);

        switch(nivel)
        {
            case 1: _delay_ms(500); break;
            case 2: _delay_ms(450); break;
            case 3: _delay_ms(400); break;
            case 4: _delay_ms(300); break;
            case 5: _delay_ms(250); break;
        }

        clearDisplay();

        _delay_ms(120);
    }

    enviarPIC(2);

    uint8_t error = 0;

    for(uint8_t i=0;i<longitud;i++)
    {
        uint8_t tecla = leerBoton();

        if(tecla != secuencia[i])
        {
            error = 1;
            break;
        }
    }

    if(error)
    {
        draw(errBmp);

        enviarPIC(3);

        _delay_ms(2000);

        nivel = 1;

        for(uint8_t i=0;i<8;i++)
            secuencia[i] = rand()%4;
    }
    else
    {
        draw(okBmp);

        _delay_ms(1000);

        if(nivel == 5)
        {
            enviarPIC(4);

            _delay_ms(2500);

            nivel = 1;
        }
    }
}
```



```

        for(uint8_t i=0;i<8;i++)
            secuencia[i] = rand()%4;
    }
    else
    {
        nivel++;
    }
}

clearDisplay();
}
}

```

Explicación del código 1:

El código del ATmega328P opera como el controlador principal del sistema. Inicializa el Puerto D para la lectura de cuatro pulsadores con pull-ups internos, el Puerto B para manejar la matriz LED mediante SPI por software y los 4 bits menos significativos del Puerto C como un bus de datos de salida hacia el PIC. La lógica del juego ahora gestiona 5 niveles de dificultad. En cada iteración, el microcontrolador genera una secuencia aleatoria de flechas direccionales y las despliega en la matriz, reduciendo el tiempo de visualización (`_delay_ms`) a medida que el nivel aumenta. La función `enviarPIC()` se encarga de la sincronización: coloca un valor de 4 bits en el bus, mantiene el dato por 100 milisegundos y lo limpia, enviando "pulsos" de comandos al esclavo para solicitar los tonos correspondientes al nivel, al inicio del turno, a la victoria o al error.

Finalmente, el programa valida las entradas del usuario con funciones anti-rebote por software y decide si avanzar de nivel o mostrar la animación de "X" y reiniciar el juego.

Código para PIC16F887 (Buzzer):

```

unsigned char anterior = 0;

void melodiaInicio()
{
    Sound_Play(523,150);
    Delay_ms(180);

    Sound_Play(659,150);
    Delay_ms(180);

    Sound_Play(784,200);
}

void turnoJugador()
{
    Sound_Play(1000,100);
    Delay_ms(120);

    Sound_Play(1000,100);
}

void melodiaError()
{
    Sound_Play(250,500);
}

void melodiaVictoria()
{
    Sound_Play(523,100);
}

```



```
    Delay_ms(120);

    Sound_Play(659,100);
    Delay_ms(120);

    Sound_Play(784,100);
    Delay_ms(120);

    Sound_Play(1046,300);
}

void nivel1()
{
    Sound_Play(400,100);
}

void nivel2()
{
    Sound_Play(700,100);
}

void nivel3()
{
    Sound_Play(1000,100);
}

void nivel4()
{
    Sound_Play(1200,100);
}

void nivel5()
{
    Sound_Play(1400,100);
}

void main()
{
    ANSEL = 0;
    ANSELH = 0;

    TRISB = 0xFF;

    TRISC2_bit = 0;

    Sound_Init(&PORTC,2);

    while(1)
    {
        unsigned char actual;

        actual = PORTB & 0x0F;

        if(actual != anterior)
        {
            switch(actual)
            {
                case 1:
                    melodiaInicio();
                    break;

                case 2:
                    turnoJugador();
                    break;

                case 3:
                    melodiaError();
```



```
        break;

    case 4:
        melodiaVictoria();
        break;

    case 5:
        nivel1();
        break;

    case 6:
        nivel2();
        break;

    case 7:
        nivel3();
        break;

    case 8:
        nivel4();
        break;

    case 9:
        nivel5();
        break;
    }

    anterior = actual;
}
}
```

Explicación del código 2:

El PIC16F887 funciona como un periférico esclavo dedicado exclusivamente a la síntesis de audio, utilizando la librería Sound nativa de MikroC para generar señales PWM a través del pin RC2. Se configuran los pines analógicos como digitales (ANSEL = 0) y el Puerto B completo como entrada. El programa se basa en un bucle infinito que lee constantemente los 4 bits inferiores del bus de datos (PORTB & 0x0F).

Su esquema de sincronización es por detección de cambio de estado: el microcontrolador compara el valor actual del puerto con el valor leído en el ciclo anterior (actual != anterior). Si detecta un cambio válido, la estructura switch-case decodifica el comando numérico (del 1 al 9) y ejecuta la función de audio asociada. Esto le permite reproducir desde tonos simples para identificar en qué nivel (1 al 5) se encuentra el jugador, hasta melodías compuestas secuenciales para los eventos de inicio de turno, fracaso y victoria.

4. Descripción del esquema de comunicación de microcontroladores

El sistema utiliza un bus de datos digital paralelo unidireccional de 4 hilos, operando bajo una arquitectura Maestro (ATmega328P) a Esclavo (PIC16F887).

1. Capa Física

La conexión se establece directamente empleando un *nibble* (4 bits) del puerto de salida del ATmega y el puerto de entrada del PIC:

- Transmisor: Pines PC0 a PC3 del ATmega328P.

- Receptor: Pines RB0 a RB3 del PIC16F887.

Al utilizar 4 bits, el bus tiene la capacidad teórica de transmitir hasta 16 comandos distintos (2^4), lo cual cubre perfectamente los 9 eventos del juego.

2. Lógica de Sincronización (Cambio de Estado)

A diferencia de protocolos que usan un pin extra de reloj o *strobe*, esta versión utiliza sincronización por detección de flanco de bus (cambio de estado) y retorno a cero:

- El Maestro coloca el código en el bus (`PORTC = ... | dato`), mantiene ese estado lógico durante exactamente 100 milisegundos (`_delay_ms(100)`) para asegurar que el receptor lo lea, y finalmente limpia el bus (`PORTC &= 0xF0`). Este "retorno a cero" es crucial, ya que permite que el PIC detecte comandos consecutivos repetidos.
- El Esclavo realiza un escaneo continuo (*polling*) leyendo los 4 bits inferiores del puerto B. Utiliza la lógica `if(actual != anterior)` para disparar la acción de sonido solo cuando detecta una variación respecto al último estado leído, actuando como un filtro eficaz contra ruidos y repeticiones no deseadas.

3. Diccionario de Datos

Los valores binarios transmitidos corresponden a comandos acústicos precisos, definidos en la siguiente tabla de verdad:

Comando (Decimal)	Pines (PC3:PC0 → RB3:RB0)	Acción Ejecutada en el PIC16F887
0	0000	Reposo (Estado de limpieza del bus)
1	0001	Melodía de inicio de secuencia de flechas
2	0010	Tono doble indicador de "Turno del jugador"
3	0011	Melodía de Error (Derrota)
4	0100	Melodía de Victoria (Juego superado)
5	0101	Tono indicador de Nivel 1
6	0110	Tono indicador de Nivel 2
7	0111	Tono indicador de Nivel 3
8	1000	Tono indicador de Nivel 4
9	1001	Tono indicador de Nivel 5

5. Enlace de repositorio GitHub

Repositorio de GitHub: <https://github.com/ivagbald/SimonDice-SistEmbebidos.git>

6. Conclusiones y recomendaciones

Conclusiones:

- La implementación de un bus de datos paralelo de 4 bits con lógica de sincronización por cambio de estado (actual != anterior) demostró ser altamente eficiente para la arquitectura Maestro-Esclavo. Este método eliminó la necesidad de una línea de reloj o *strobe* dedicada, minimizando la latencia y garantizando una sincronización precisa entre las animaciones de la matriz LED (ATmega328P) y las melodías generadas (PIC16F887).
- El desarrollo de rutinas de transmisión serial por software para controlar el driver MAX7219 validó la flexibilidad del ATmega328P. Al no depender estrictamente de los pines del hardware SPI nativo, se logró un control total sobre el despliegue dinámico de matrices de bytes (flechas, números y retroalimentación visual) optimizando la asignación de pines del microcontrolador.
- Se concluye que la lectura de ruido analógico (flotante) a través del módulo ADC para alimentar la semilla generadora de números aleatorios (`srand(ADC)`) es un requisito fundamental en el diseño de juegos electrónicos. Esto resolvió la limitación de las secuencias predecibles, asegurando que los 5 niveles del juego presenten un desafío de memorización verdaderamente aleatorio en cada ejecución.

Recomendaciones:

- Aunque el código del ATmega328P incluye rutinas de *debounce* por software (`_delay_ms(50)` y bucles `while`), se recomienda encarecidamente añadir filtros pasivos RC (Resistencia-Capacitor) o compuertas Schmitt Trigger en los pulsadores físicos. Esto descargará al microcontrolador de tareas de validación y evitará falsos positivos producidos por el desgaste mecánico de los botones.
- Para futuras iteraciones o escalabilidad del proyecto, se sugiere reemplazar el bus paralelo de 4 hilos por un protocolo de hardware dedicado como I2C o UART. Esto liberará pines I/O en ambos microcontroladores, permitiendo la integración de nuevos periféricos, como pantallas LCD para llevar un registro de puntajes o sensores adicionales.